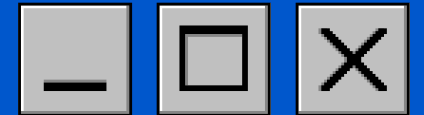


Projeto System Call

NO XV6

Monitoramento e Escalonamento





INTEGRANTES



CARLOS EDUARDO



DAVI GONÇALVES



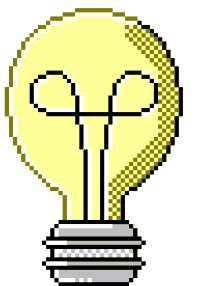
LEONARDO PEREIRA

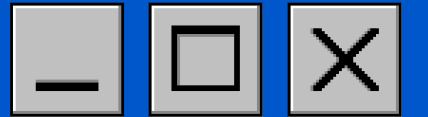


MATEUS GOMES



YASMIN MOREIRA





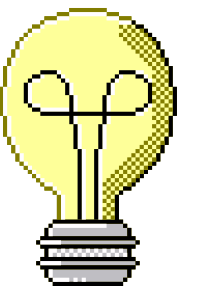
System Call: `getpinfo()`

O que é?

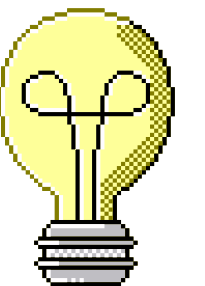
- Uma chamada de sistema didática (comum no SO xv6) que inspeciona a tabela de processos do Kernel e retorna o estado atualizado do sistema para o usuário.

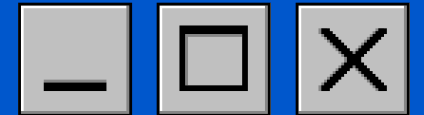
Qual é o seu propósito?

- Atuar como a base para criar um comando de monitoramento (como o `ps`).
- Permitir a auditoria e visualização do comportamento do escalonador de processos (Scheduler).



- Exploração de Baixo Nível
- Visibilidade e Controle do Sistema
- Round Robin
- Structs para pacotes de dados





ALGORITMOS E MECANISMOS IMPLEMENTADOS



Alterações do S0 XV6 (Kernel/User)

Implementação da Syscall (getpinfo)

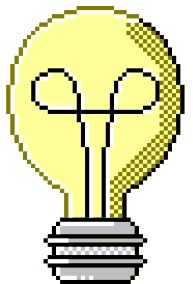
- `sysproc.c`: implementação da função `getpinfo(struct*)`
- `syscall.c`: adicionar entrada na tabela de syscalls
- `syscall.h`: definir número da syscall
- `usys.pl`: adicionar gatilho para chamada do usuário
- `user.h`: declarar protótipo da função `getpinfo(struct*)`

Implementação do Escalonador Round Robin Prioritário

- `proc.c`: Alterar a função `scheduler()` com nossa nova implementação
- `proc.h`: adicionar campo de prioridade na estrutura de processos

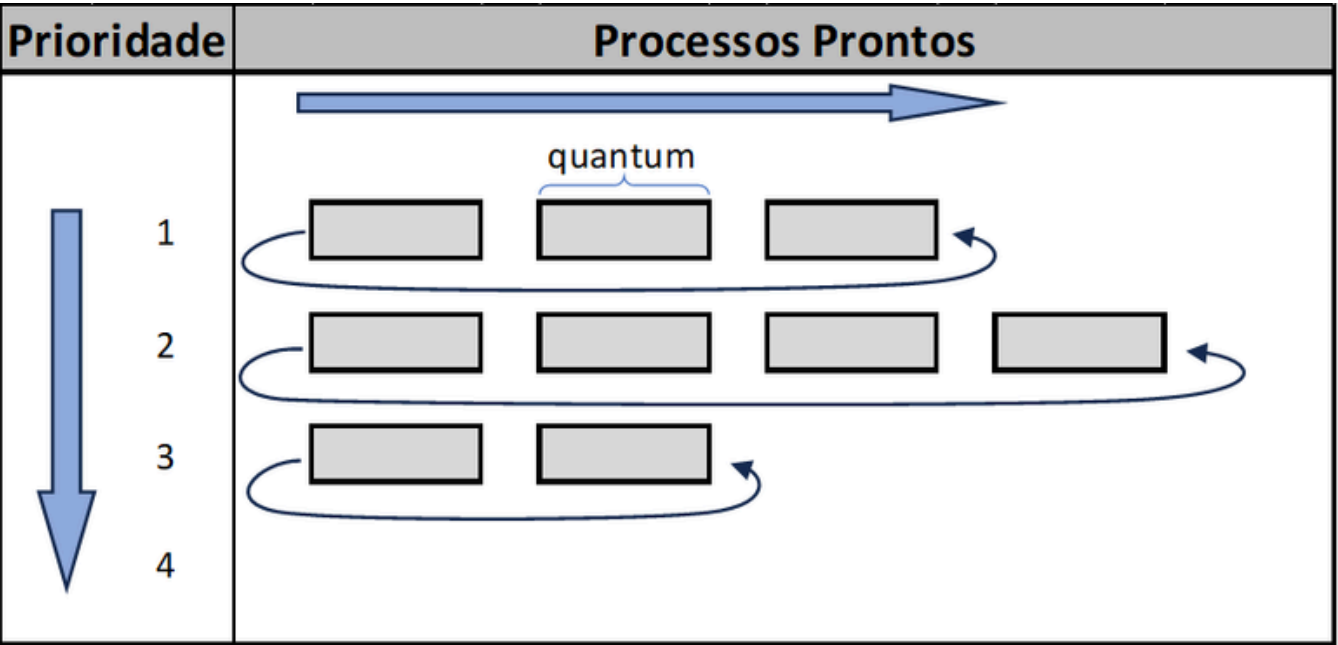
Programas de usuário para teste

- `geraProc.c`: gera processos ativos no sistema para teste
 - Fibonacci recursivo com valores variados (?)
- `testepInfo.c`: chama `getpinfo()` e imprime PID, prioridade, estado, ticks.

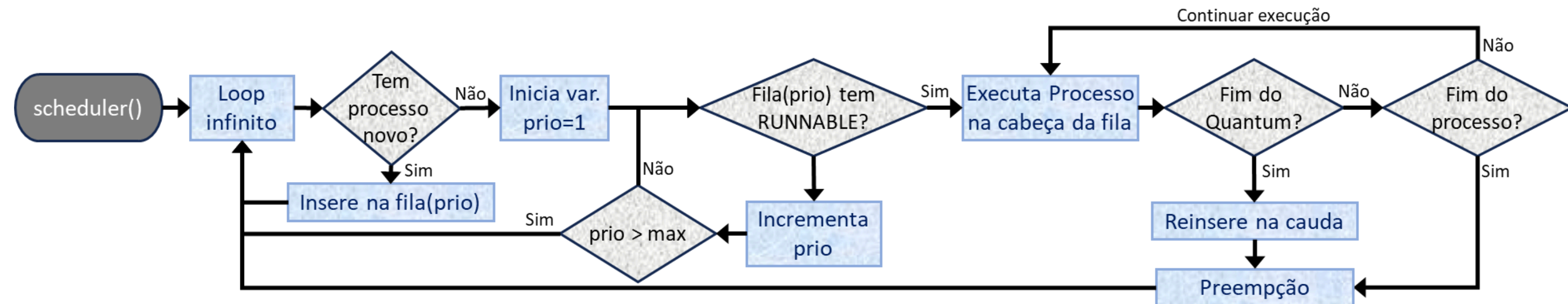


Round Robin Prioritário

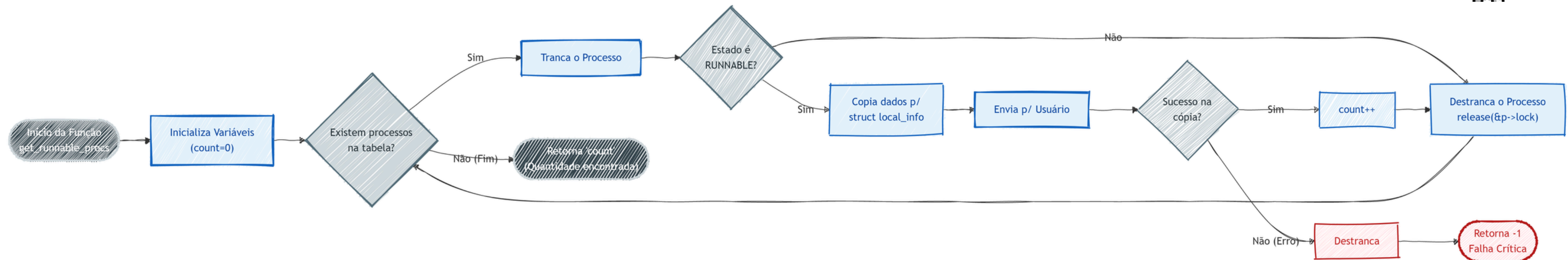
O escalonador divide os processos em filas de prioridade nas quais o Round-Robin tradicional será aplicado. O Round-Robin, por sua vez, atribuirá a cada processo uma parcela de tempo, o quantum, que será o indicador de quanto tempo um processo pode ficar na CPU. O algoritmo, assim, continuará até que a lista de processos prontos esteja vazia.



Round Robin Prioritário

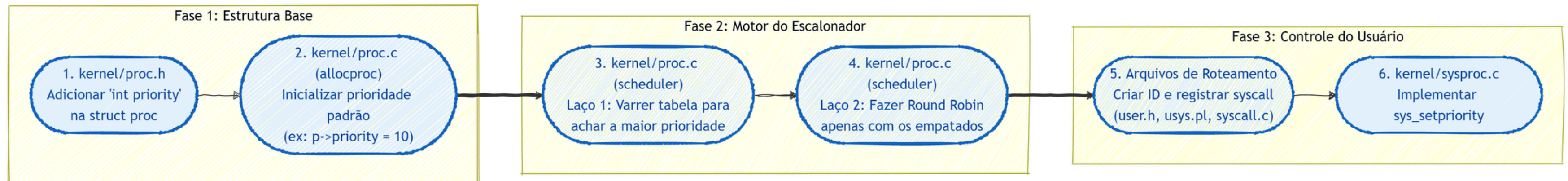


System Call getpinfo(struct*)



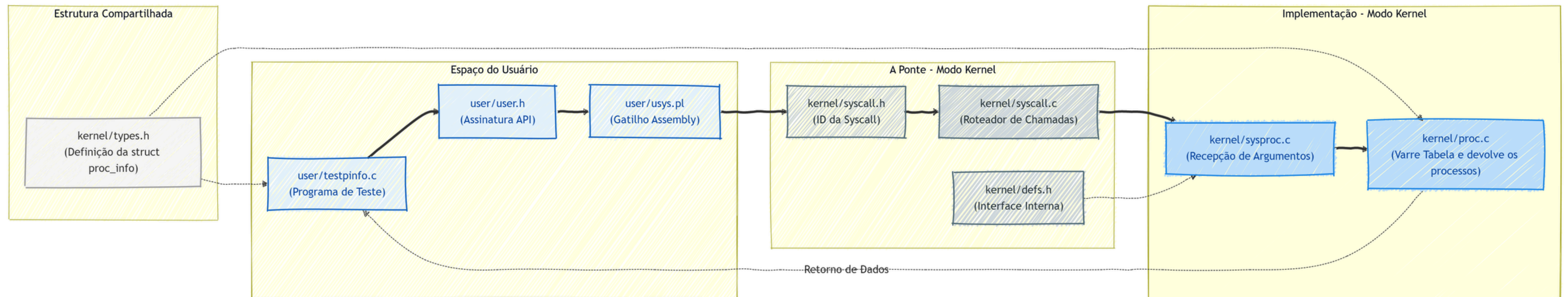


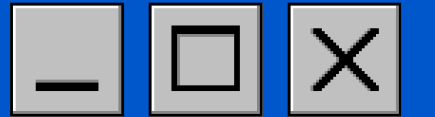
Round Robin Prioritário





System Call `getpinfo(struct*)`





CONCLUSÃO

- **Kernel Eficiente e Justo:** Esperamos um sistema com distribuição precisa de CPU e sem processos invisíveis.
- **Auditoria em Tempo Real:** A `getpinfo()` permitirá validar visualmente o comportamento do escalonador.
- **Teoria na Prática:** O projeto consolidará nossa compreensão sobre o controle e o funcionamento de baixo nível.

Ok

